AI GATEWAY · PROMPT COMPRESSION & CONTEXT MANAGEMENT

Does compressing the prompt hurt routing?

Six configurations of our module, run against Sajjad's LLM-as-judge routing baseline on RouterBench. Same judge, same dataset, same ground truth — the only thing that changes is what we did to the prompt before the judge read it.

`~1,600 judge calls` `1 error` `judge: gpt-5.1-nano @ temp 0`
`ground truth: RouterBench, 36,481 rows` `100 prompts per arm` `2026-08-28`

Half the tokens. Same accuracy.

On long prompts, our shipping configuration — unmodified, nothing forced — compressed 95 of 100 prompts and landed on the baseline's accuracy exactly.

−47.6%	±0.00pp	95/100
JUDGE INPUT TOKENS	ROUTING ACCURACY	PROMPTS COMPRESSED

ALL FOUR STAGES

What each part of the module actually did

canonicalize — SHIP IT ON

0% → 76%

Tokens served from the judge's cache. Same content, same token count — only the order changed.

2.5 ms · cannot alter text

compress — KEEP THE GATE

−47.6%

Judge input tokens on long prompts, at zero measurable accuracy cost. Declines short prompts, correctly.

~640 ms · fires above 5,000 tokens

summarize PROMISING

−81.4%

Session tokens on 33-message chats. Question and recent four turns kept verbatim in 25/25.

3.5 s · once per session · quality untested

rewrite LEAVE OFF

80–90%

Share of prompts it rejected its own output on. The drift guard is right — there is nothing here to clarify.

1.1–1.4 s per prompt, for nothing

STUDY 1 · LONG PROMPTS

Synthetic RAG, ~6,000 tokens per prompt

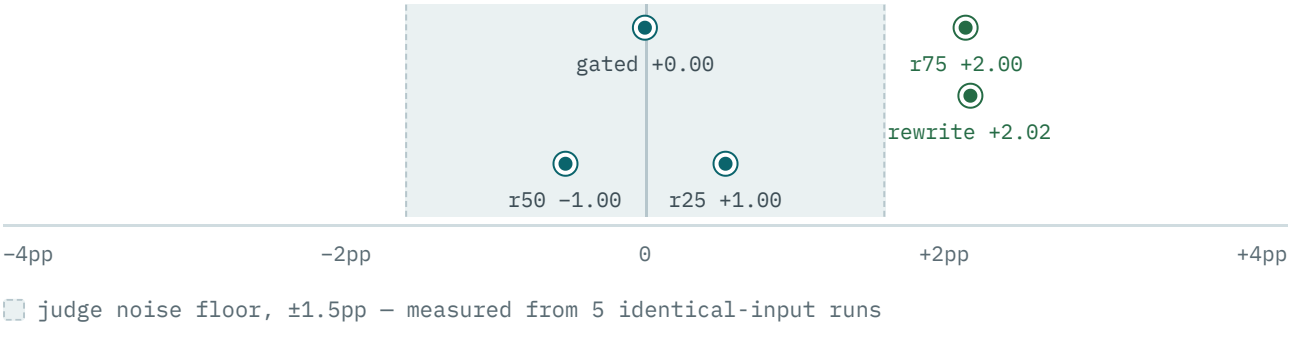
RouterBench has no long prompts, so each question was wrapped in retrieved context. The compressor was pointed at the *context* only — the question is labelled and protected.

ARM	ACCURACY	Δ	95% CI	JUDGE TOKENS / CALL	RE-ROUTED	PCM
baseline	61.25%	–	–	6,307	–	0 ms
gated OUR DEFAULT	61.25%	+0.00pp	[−4.0, +4.0]	3,303	20%	727 ms
compress_r75	63.25%	+2.00pp	[−2.0, +6.0]	4,886	23%	634 ms
compress_r50	60.25%	−1.00pp	[−7.0, +5.0]	3,194	23%	637 ms
compress_r25	62.25%	+1.00pp	[−2.0, +4.0]	1,861	25%	635 ms
rewrite	63.13%	+2.02pp	[−2.0, +6.1]	6,303	18%	1,083 ms

HOW TO READ THOSE DELTAS

No arm beat the baseline measurably

The judge is **not deterministic** at temperature 0. Five runs on byte-identical prompts spread 1.50pp and re-routed 2–7 of 100 prompts on their own. Three arms sit inside that band outright. The two that do not — both about +2pp, in our favour — have confidence intervals spanning zero, so they are not established gains either. The honest claim is *“quality does not measurably degrade”*, not *“quality improved”*.



STUDY 2 · CACHE REUSE

The cheapest win in the whole study

The canonicalizer rewrites nothing — it only puts stable content first and volatile content last. Identical bytes, identical token count, 30 calls each.

canonical order — 28/30 CALLS HIT

76.1%

57,600 of 75,652 tokens served from cache.

`stable prefix first, volatile last`

scrambled order — 0/30 CALLS HIT

0.0%

0 of 75,652 tokens served from cache.

`volatile content first`

A false start worth recording: the first run measured 0% for *both* orderings and nearly wrote the stage off. The judge prompt was 519 tokens — below the provider's cache minimum. Below that floor ordering is irrelevant; above it, ordering is everything.

STUDY 3 · SHORT PROMPTS

RouterBench as it ships — and why it cannot test this

All 36,481 prompts are under 2,000 tokens (median 178, max 1,026). Compression pays above ~5,000. **Our default declined all 100 prompts, correctly** — every number below came from forcing it past its own gate.

ARM	ACCURACY	Δ	RE-ROUTED	JUDGE TOKENS / CALL	JUDGE LATENCY
baseline	62.25%	—	—	519	2.03 s
gated DECLINED ALL	60.75%	−1.50pp	7%	519 (0%)	1.92 s
compress_r75	59.75%	−2.50pp	7%	472 (−9%)	2.32 s
compress_r50	61.25%	−1.00pp	18%	414 (−20%)	2.15 s
compress_r25	61.75%	−0.50pp	16%	370 (−29%)	2.51 s
rewrite	61.25%	−1.00pp	9%	513 (−1%)	2.24 s

Fewer tokens did not mean a faster judge. `compress_r25` sent the fewest tokens and was the slowest arm. At ~500-token inputs, latency is fixed overhead — compression bought nothing and cost 41 ms of our own. Note also that the token saving is a third of the prompt-level figure: the judge also reads a fixed 330-token system prompt and model-description block, so 55% off the prompt is 20% off the bill.

SHORT VS LONG · SIDE BY SIDE

Prompt length is the whole story

The same module, the same settings, the same judge — run against a 178-token prompt and a 5,990-token one. Everything that matters flips.

	SHORT · NATIVE ROUTERBENCH	LONG · SYNTHETIC RAG
median prompt	178 tok	5,990 tok
our gate fired on	0 / 100	95 / 100
judge tokens — our default	519 (0%)	3,303 (−47.6%)
judge tokens — forced r50	414 (−20%)	3,194 (−49.4%)
accuracy Δ — our default	−1.50pp (declined all)	+0.00pp
re-routed — our default	7% (= noise)	20%
pcm cost per prompt	41 ms	~640 ms
judge got faster?	no — slower	no change
verdict	NOT WORTH RUNNING	WORTH RUNNING

Why the same setting saves 20% on one and 49% on the other. Forced `compress_r50` cuts about 55% of the *prompt* in both cases. But the judge also reads a fixed 330-token scaffold on every call. On a 178-token prompt that scaffold is most of the input, so halving the prompt barely dents the bill. On a 5,990-token prompt the scaffold is noise, and the prompt-level saving passes through almost intact. This is the mechanism behind the 5,000-token gate, measured rather than assumed.

One thing not to read across the columns: the two accuracy figures come from different datasets, so 61.25% on the left and 61.25% on the right are a coincidence, not a match. Only the *behaviour* compares — whether the gate fires, what it saves, what it costs.

BEFORE ANYONE QUOTES THESE NUMBERS

What the study does not show

! **The long-prompt and multi-turn sets are synthetic**

RouterBench has neither. Distractor passages and prior turns are other RouterBench questions — topically unrelated, which is the *easy* case for both compression and summarization. Real retrieval returns similar passages, which is materially harder. Read −47.6% and −81.4% as upper bounds.

! **n = 100 cannot resolve these effect sizes**

Confidence intervals are ~±4pp wide on ~1pp differences. “No detectable change” means exactly that — not “no change”.

! **This measures routing decisions, not answer quality**

RouterBench supplies pre-computed per-model scores; no candidate model was ever called. Whether a compressed prompt yields a worse *answer* is a different experiment, not yet run.

! **Downstream cost rose slightly, and that is not a saving**

Re-routing sent some prompts to pricier models ($0.000257 vs $0.000217 per query). Indistinguishable from noise at this sample size — but it should not be reported as a win.

WHAT TO DO NEXT

Recommendations

1 **Ship the canonicalizer on by default**

Free, cannot damage text, and the only effect that clears the noise floor decisively. 0% → 76% cache reuse.

2 **Keep `compressor.min_tokens = 5000`**

Both studies support it. Below the gate compression buys nothing and costs latency; above it, half the tokens for no measurable accuracy cost.

3 **Leave the rewriter off for this traffic**

It declines 80–90% of prompts at over a second each. The guard is working; the stage simply has no job on exam-style questions.

4 **Re-run on real long-context traffic**

The −47.6% comes from synthetic RAG with unrelated distractors. That is the experiment that would turn `min_tokens` from a literature-derived default into a measured one.

SIDE EFFECT

Two bugs the benchmark found that unit tests missed

a **The code detector matched ordinary English**

`from`, `return`, `class`, `public` are normal words — any two in a paragraph flagged the text as code and silently disabled compression. It mislabelled **28 of 100** plain multiple-choice questions. 95 unit tests missed it because the fixtures were real code and real prose, never prose containing the word “from”. Now 0/100.

b

Duplicate force-tokens crash LLMLingua

Passing the same PII placeholder repeatedly fails with a bare `AssertionError` — survives 60 repeats, dies at 120, exactly the prompt size the stage exists for. Only real weights could have caught it.
